Media processing architecture overview

Table of contents

- Introduction
- The media processing unit (MPU)
- Media processing unit (MPU) anatomy
  - The transcoding module
  - Transcoding plugins
  - Plugin selection
- Deployment options
  - The transcoding protocol
  - Multiple clients sharing an MPU
  - Multiple MPUs per client

Introduction

Media processing (also called *transcoding*) is the manipulation of raw, uncompressed media data. In general, this involves the following steps:

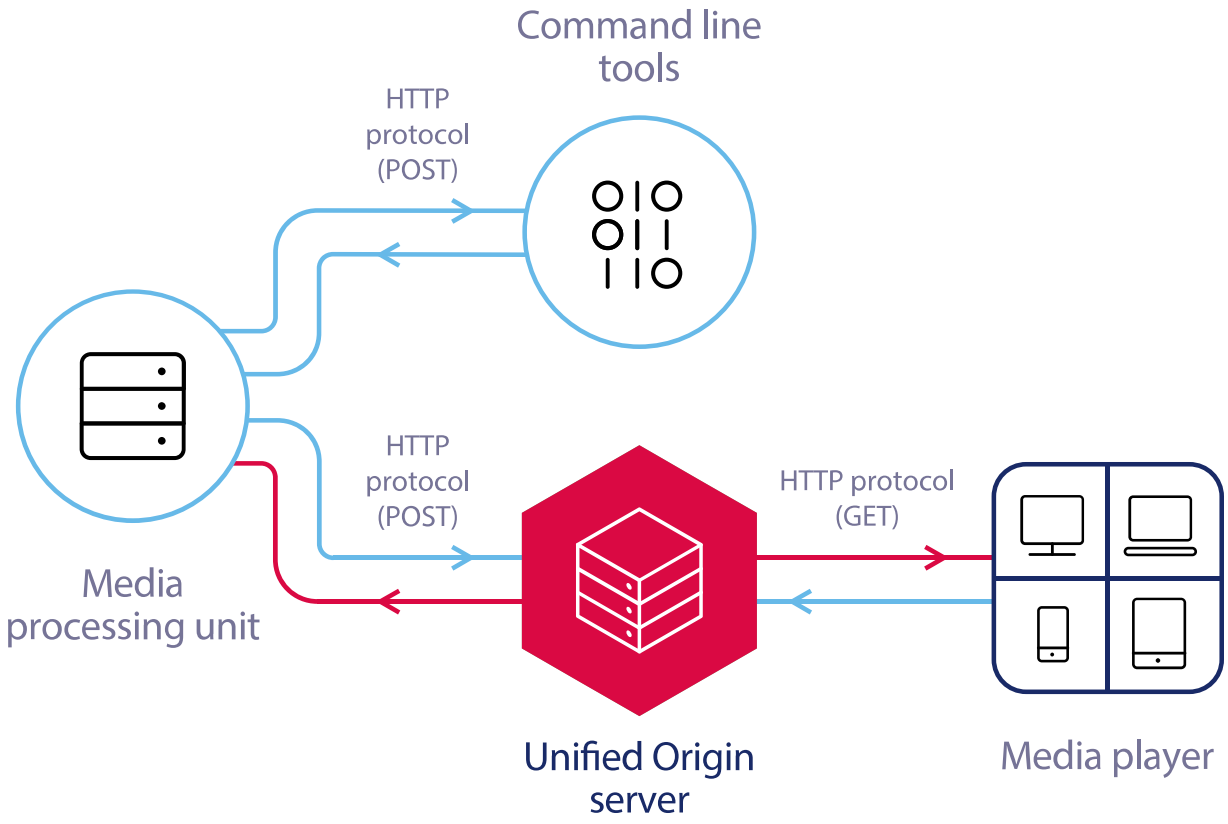
- decoding (*decompression*): compressed samples are translated to some non-compressed format, such as raw video frames.
- manipulating the stream of uncompressed data.
- re-encoding (*compression*) of the uncompressed data, possibly to a different compression format.

The media processing step is usually part of a larger workflow initiated from one of our command line tools, such as the static packager (`mp4split`), `unified_remix`, or `unified_capture`. Example use cases include keyframe insertion (for frame-accurate clipping), resizing video frames, and generating jpeg images from a video track.

Media processing can be achieved either locally, with transcoding happening on the same host of the client application, or using a dedicated media processing unit (MPU), a Web Application installed and configured separately that clients access remotely via HTTP.

Local transcoding must be considered a legacy workflow: it is not recommended for customers that must configure media processing for the first time. Existing local-transcoding workflows should be migrated to make use of a media processing unit. At some point, in the long term, support for local transcoding will be officially deprecated and eventually removed.

The media processing unit (MPU)



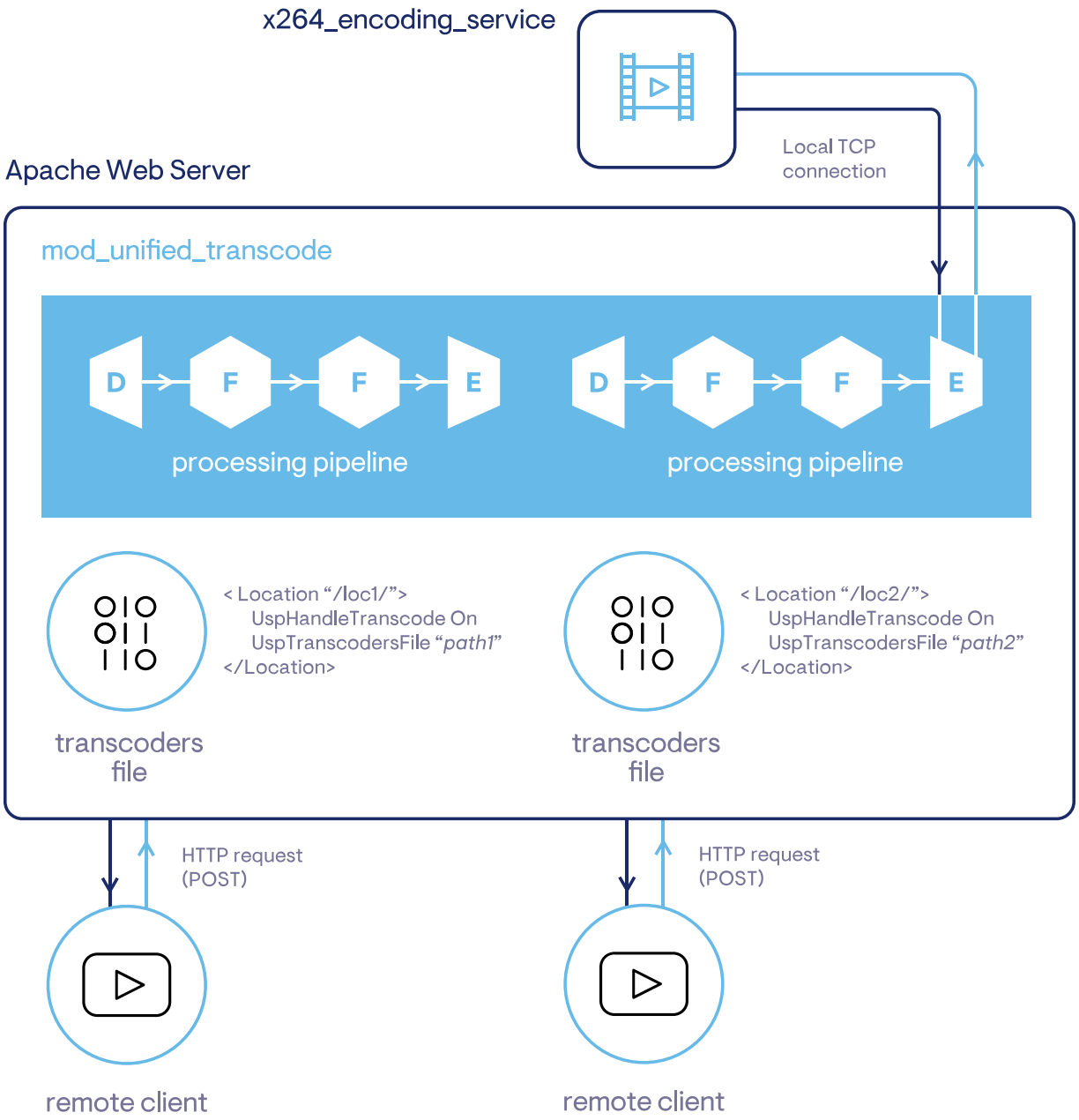
Note

The above image includes Unified Origin for illustrative purposes only. Usage of the media processing unit is at the moment limited to command line tools only.

The operations involved in media processing (decoding, raw data manipulation, and re-encoding) may consume significant amounts of computing power, which can affect the performance of other software components running on the same host. Furthermore, your deployment options may be constrained by requirements on the hardware or the operating system, the availability of specific runtime libraries, and issues related to patent licensing.

For these reasons, we recommend the use of an appropriately configured dedicated (virtual) server: the **media processing unit**, which is deployed in close proximity to the origin server.

Media processing unit (MPU) anatomy



The transcoding module

The media processing unit uses an Apache Web Server that, for some of its URLs, is configured to provide media processing operations in the form of HTTP POST requests by calling into Unified Streaming's `mod_unified_transcode` module.

To handle a media processing request, `mod_unified_transcode` builds and then executes a *processing pipeline*. The pipeline consists of a decoder instance (marked by a **D** in the image above), zero or more filter instances (**F**), and an encoder instance (**E**).

Transcoding plugins

Based on its function, each item in the processing pipeline has a specific *transcoder type* (e.g. `video_decoder_avc`, `video_filter_resize`, or `video_encoder_jpg`). For many transcoder types, multiple implementations are available, which are provided in the form of dynamically loaded *transcoding plugins*. In most cases, we provide a plugin that uses the FFmpeg libraries, and one that uses the Intel Media SDK.

For AVC encoding (the `video_encoder_avc` transcoder type), users have a choice between a plugin that calls into a dedicated x264_encoding_service process over a local TCP connection, and a plugin that uses the Intel Media SDK.

The full list of available video transcoding plugins can be found here.

> **Note**
>
> For Linux, the Intel Media SDK plugins are only supported on Ubuntu 20.04 or later. Access to an Intel GPU is required; this implies that these plugins cannot be used in most containerized environments. See here for further details.

> **Attention**
>
> Use of the Intel Media SDK will become deprecated in the future. It will be replaced with the use of the FFmpeg Libraries and the x264 encoding service.

> **Attention**
>
> Please be aware Media Processing Unit contains the same capabilities as the IntelMediaSDK and does not support video encoding for hevc therefore the following error response will be generated.
>
> `FMP4_NOT_IMPLEMENTED video encoder for codec hvc1 not implemented`
>
> This can be mitigated by using Using dynamic track selection by filtering out the track from the source using `filter=(FourCC!="hvc1")'`.

Plugin selection

To determine which plugins to load, `mod_unified_transcode` uses a transcoders file. For each pluggable transcoder type, this file contains a line that specifies the plugin to be loaded, including any plugin-specific attributes. The path to the transcoders file is listed in the Apache

configuration.

In the image above, Apache is configured to handle transcoding requests for URL paths starting with either `/loc1/` or `/loc2/`. Each of these locations has its own transcoders file, with a (possibly) different set of selected plugins and plugin-specific attributes.

Deployment options

In this section, we show how the use of one or more dedicated media processing units, as opposed to local transcoding, can provide scalability in dealing with transcoding workloads that may be hard to estimate in advance.

The transcoding protocol

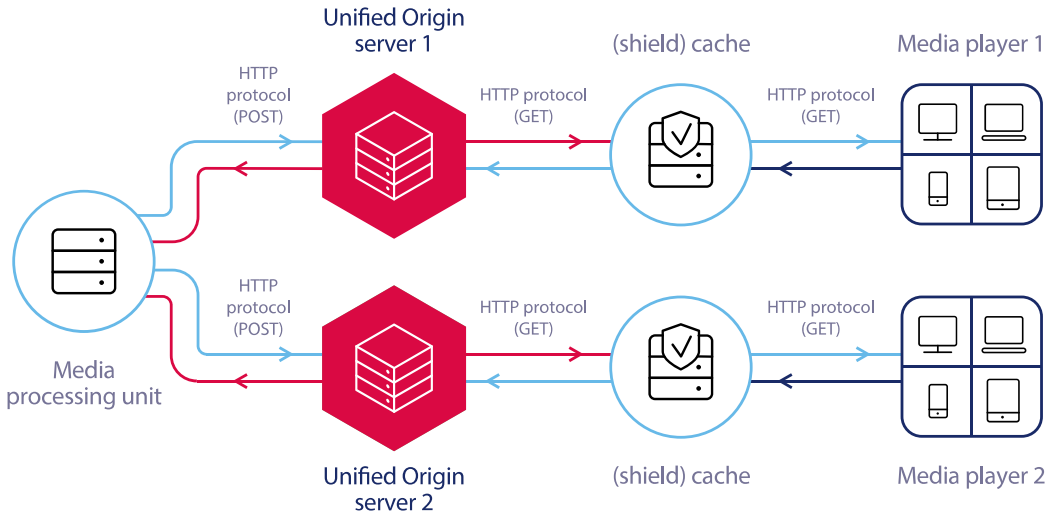
The protocol used between the transcoding client (such as a command-line application or a Unified Origin server) and the media processing unit uses a single HTTP POST request for each transcoding operation.

On the media processing unit side, the requested operation is handled by first constructing a processing pipeline. This pipeline is then used to transcode an arbitrary, and potentially large, amount of media data. The data to be transcoded is sent in the request body, and the transcoded data is returned in the reply body.

In both directions, the protocol uses HTTP chunked transfer encoding. To reduce both latency and memory pressure, the media processing unit will start returning chunks of transcoded data as soon as they become available, even *before* the end of the request body is received. In other words: during the transcoding operation, the HTTP connection between the transcoding client and the media processing unit is used as a fully bidirectional pipe.

> **Note**
>
> Under these conditions, request-level caching between the transcoding client and the media processing unit makes little sense: it would just lead to high memory pressure on the cache and extra latency for the client.

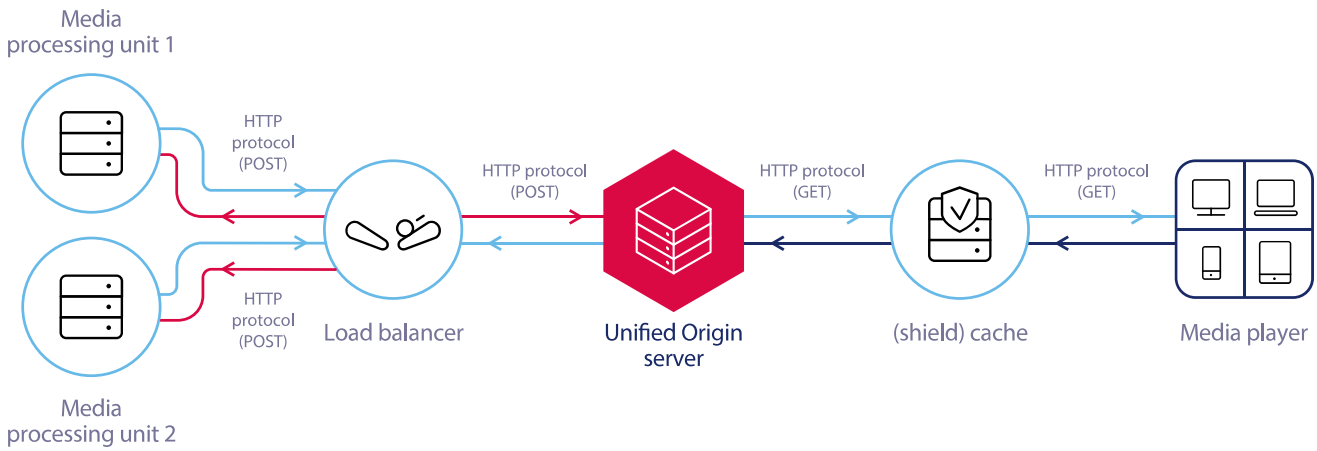
Multiple clients sharing an MPU



This is an appropriate lightweight deployment when the transcoding clients are usually busy handling other tasks, and transcoding is only used occasionally. Its main advantage is simplicity: only a single media processing unit (with a single media processing license) is required.

Please note that any request-level caches are placed between the media players and the origin servers: the potential for reduced latency applies to all GET requests to the origin servers (including those that require transcoding).

Multiple MPUs per client



This is the opposite of the deployment described above, and applies when the transcoding clients heavily rely on transcoding, spending most of their time waiting for transcoded media data.

A request- or connection-level *load balancer* is used to distribute the transcoding workload over multiple media processing units; its main advantage is the ability to scale up by simply adding another media processing unit. Please note, however, that each media processing unit requires its own media processing license.

Again, request-level caches, if any, are placed between the media players and the origin servers, for the same reason as described for the deployment above.